

Day 1 — Foundations: Full Instructor Teaching Material (Technical Edition)

Total session time: 210 minutes (3.5 hrs). Each section is sized to its time budget with talking points, board work, wiring/ladder detail, and explicit framing lines.

Addressing convention used throughout (adjust to whatever platform the lab runs — Allen-Bradley/RSLogix, Siemens TIA/S7, or CODESYS/IEC 61131-3 generic):

- IEC generic: `%I0.0` (input bit), `%Q0.0` (output bit), `%M0.0` (internal memory bit)
- Allen-Bradley: tag-based, e.g. `Start_PB`, `Motor_Run`
- Siemens: `I0.0`, `Q0.0`, `M0.0`

1. PLC Basics — 15 min

Definition, precisely

A PLC is a purpose-built industrial computer executing a single control program in a deterministic, cyclic loop, reading digital/analog inputs and driving digital/analog outputs each cycle. Core architecture to put on the board:

```
[Power Supply] --24VDC--> [CPU] <--backplane bus--> [Input Modules] <-- field wiring -- sensors/switches
|
[Output Modules] -- field wiring --> actuators/lamps/motors
```

Why it replaced relay panels — the technical distinction, not just the analogy

A relay panel implements Boolean logic in *hardware*: each relay coil, when energized, physically closes or opens a set of contacts wired to the next relay or output. Changing logic means physically re-wiring contacts across a rack that can span dozens of relays for a moderately complex machine.

A PLC's CPU executes the equivalent Boolean logic as *stored program instructions* against an input/output image table held in RAM. The physical I/O wiring topology to field devices does not change when logic changes — only the program in memory changes, which can be edited and downloaded to the CPU in minutes.

Analogy: house wiring vs. software — same physical switches and lights, but behavior lives in an editable program instead of physical connections.

Real-world use cases — conveyor belts (start/stop, jam detection via a timed no-motion condition on a speed sensor), packaging lines (fill/seal/count cycles), elevators (floor sequencing with door interlocks — a direct preview of interlocking logic in today's end-of-day project), traffic lights (state-sequenced timing — the entire subject of Day 2's Day Project, worth flagging as forward reference).

Key terms, defined with the precision they'll need for the rest of the week:

- **Input** — a physical signal (digital or analog) read into the CPU's input image table each scan

- **Output** — a physical signal written from the CPU's output image table to a physical terminal each scan
- **Program/Logic** — the stored set of instructions (ladder, structured text, function block) solved each scan against the frozen input image
- **Scan** — the word only for now; the full input-scan/logic-solve/output-update cycle is unpacked in Block 3

Common misconception to correct explicitly: a PLC is not a general-purpose multitasking OS. It runs one control program in a tight, predictable, repeating loop, hardened against vibration, temperature extremes, and electrical noise (via features like galvanic isolation on I/O modules and conformal-coated boards) — a general-purpose PC is not designed or certified for this environment.

2. PLC Hardware — 30 min

2a. CPU & Power Supply — 8 min

Power supply, with actual voltage specifics: converts mains AC (100–240V, 50/60Hz, depending on region) to 24V DC, the near-universal standard for PLC I/O-side power. State explicitly why 24V DC specifically:

- Low enough that accidental contact is not lethal under normal conditions (compare to mains voltage), a meaningful safety margin for technicians working live on a panel
- Standardized across virtually every industrial sensor/actuator manufacturer, meaning a 24V DC proximity sensor from one vendor plugs directly into a PLC input module from a different vendor with no voltage-matching concerns
- Low enough voltage drop tolerance issues are manageable at typical field wiring run lengths compared to lower voltages like 5V/12V, while still being safely below mains

CPU executes the input-scan/logic-solve/output-update loop (full detail in Block 3) and typically has its own onboard diagnostic LEDs (RUN/STOP/FAULT/FORCE — the FORCE indicator becomes directly relevant on Day 2, Block 3b).

2b. Input Modules — 8 min

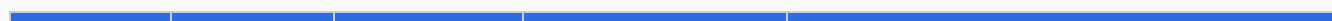
Digital input module electrical behavior, stated precisely: reads 24V DC present at a terminal as logic "1" (true), 0V (or below a defined threshold, commonly ~5V for a "0" and ~15V for a guaranteed "1", with an undefined region between) as logic "0". This threshold-based reading, not a clean binary switch, is why proper wiring practice matters — a marginal voltage due to a poor connection can produce unreliable/chattering logic state.

State explicitly: the module reports raw voltage-derived state — it has no knowledge of whether that terminal is wired to a Normally Open or Normally Closed field switch. That interpretation is entirely a function of field wiring choice (fully explored in Block 4d) plus how the programmer writes the logic against that bit.

Digital vs. analog — name only today: analog inputs read a continuous voltage/current range rather than a two-state signal; this gets a full day of coverage on Day 3.

2c. Output Modules — 8 min

Three output types, with the electrical distinctions that actually matter on the job:



Type	Switches	Typical response time	Mechanism	Notes
Relay	AC or DC	Slower (ms range, mechanical)	Electromechanical contact	Robust, galvanically isolates PLC from load, has finite mechanical switching life (typically rated in switching cycles, e.g., 10^5 – 10^6)
Transistor	DC only	Fast (μ s–ms range)	Solid-state switching	No mechanical wear, suited to high-frequency switching (e.g., PWM-style control), cannot switch AC loads
Triac	AC only	Fast, solid-state	Solid-state AC switching	Common for lamps/solenoids on AC circuits, no mechanical wear

Practical takeaway to state explicitly: if a spec sheet says "transistor output module," that module physically cannot drive an AC contactor coil — attempting to do so does not simply fail safely, it can damage the output stage. Module type selection must match the load's electrical requirements before wiring, not after.

2d. Label Exercise — 6 min

Hand out a printed PLC panel diagram. Have students physically locate and label: CPU, power supply, input terminal block, output terminal block. Keep it low-stakes and purely spatial — the goal is physical familiarity with a real panel layout before anything gets wired in Block 4.

3. PLC Scan Cycle — 15 min

The core loop, with timing specifics

State the stakes explicitly up front: this is the single most load-bearing mental model for all debugging this week.

1. INPUT SCAN — read all physical input terminals, latch values into Input Image Table (IIT) (inputs frozen here — no re-read until next cycle)
2. LOGIC SOLVE — execute entire program top-to-bottom against the frozen IIT, write results to Output Image Table (OIT) — **NOT** to physical terminals yet
3. OUTPUT UPDATE— write entire OIT to physical output terminals, all at once
[+ housekeeping: communications servicing, diagnostics, etc., platform-dependent]
--> repeat

Typical scan time: commonly in the low single-digit milliseconds for a small-to-medium program (e.g., 1–10ms), though this varies significantly with program size, instruction complexity, and CPU class. State explicitly: scan time is not fixed to an exact constant on most platforms — it varies slightly cycle to cycle depending on which logic branches execute (e.g., conditional blocks that are skipped some cycles).

Why scan time matters, quantified: if a physical input pulse (e.g., from a fast-moving proximity sensor) is shorter in duration than one scan time, the PLC can miss it entirely — the signal can go true and false again between two input-scan phases and never appear in the Input Image Table at all. This is a genuine, quantifiable failure mode in high-speed sensing applications, not just a theoretical caveat.

Why this matters for debugging, stated as a direct rule: any output that "didn't update when expected" is explained by one of exactly two things — either the input hadn't been re-scanned yet when you expected it to reflect a new state, or the output hadn't been written to the physical terminal yet because the logic-solve phase for that scan hadn't completed. There is no third category of "the PLC just didn't notice."

Whiteboard walkthrough: draw a single rung — one NO contact driving one coil. Trace explicitly through all three phases: phase 1 freezes the contact's physical state into IIT; phase 2 solves the rung against that frozen value, writing the coil result into OIT; phase 3 writes that OIT bit out to the physical output terminal. Annotate each phase with its approximate timing contribution if scan-time instrumentation is available on the lab platform.

4. Inputs/Outputs + NO/NC Contacts — 40 min

4a. Wiring a Push-Button Input — 10 min

Wire a push button to a digital input terminal. Cover explicitly:

- **Terminal numbering** — walk the physical terminal strip, point to the printed channel numbers matching the module's addressing (e.g., channel 0 = `%I0.0`)
- **Common vs. signal wire** — most digital input modules are sourcing or sinking type (PNP/NPN convention matters for 3-wire sensors specifically); the "common" terminal provides the reference (0V or 24V depending on sourcing/sinking configuration) that the signal wire is measured against
- **Verification before trusting the wiring** — open the software's I/O status/monitor view and physically press the button, confirming the corresponding input bit toggles in real time, before writing any logic against it. State explicitly: never assume a wire is correct without confirming it in software first.

4b. Wiring a Lamp Output — 10 min

Wire a lamp/LED to a digital output terminal. Confirm module type matches load (relay/transistor/triac per Block 2c table) — ask the class directly: "if this module is transistor-output, could this circuit use an AC-rated lamp?" (No — DC only, reinforcing 2c rather than treating it as a new isolated fact.)

4c. Live Status Monitoring — 10 min

Open the software's online/monitor view showing live I/O state (commonly a highlighted/colored indication per bit). State plainly: this is the single most-used debugging tool for the rest of the week. Require every student to navigate to it themselves at least once, unprompted, before moving on.

4d. NO vs. NC in Real Circuits — 10 min

Precise electrical definitions:

- **NO (Normally Open)** — contact open (no continuity) at rest; closes (continuity, current can flow) when actuated. Wired to a digital input, this reads logic "0" until actuated, "1" while actuated.
- **NC (Normally Closed)** — contact closed (continuity) at rest; opens when actuated. Wired to a digital input, this reads logic "1" until actuated, "0" while actuated.

Why E-stops are wired NC — the fail-safe principle, stated with the actual failure-mode logic:

If an E-stop is wired NC and any of the following occurs — the wire is physically cut, the connector is unplugged, the switch mechanism itself fails open — the input reads logic "0" (identical electrical signature to "button pressed"). The safety logic responds exactly as if someone had pressed the E-stop: the machine stops. **The wiring fault produces the same safe outcome as an intentional actuation.**

Contrast explicitly with the NO case: if an E-stop were wired NO and the wire were cut, the input reads logic "0" — electrically indistinguishable from "not pressed." A real wiring fault on a safety-critical NO-wired E-stop would go completely undetected while the machine continues running under a false assumption of safety. This is a formal, mandated industrial safety design principle (fail-safe design), not a stylistic convention — state this plainly and let it sit before moving on.

5. Ladder Logic Basics + Tag Naming — 60 min

5a. Rungs & Basic Instructions — 15 min

Rung structure, precisely: contacts (representing input conditions, evaluated left to right) on the left side of the rung, a coil (representing an output or internal bit, the result) on the right side. "Power flow" left to right represents the Boolean evaluation of the series/parallel contact network.

AND — series contacts, IEC ST equivalent alongside the ladder:

```
Ladder:  |--[ ContactA ]--[ ContactB ]----- ( Coil )--|
ST:      Coil := ContactA AND ContactB;
```

Both must be true (both physical/logical conditions closed) for current to "flow" through to the coil.

OR — parallel contacts (branches):

```
Ladder:  |--[ ContactA ]----- ( Coil )--|
         |--[ ContactB ]-----|
ST:      Coil := ContactA OR ContactB;
```

Either branch being true energizes the coil.

NOT — represented by using an NC-style contact symbol on the rung for that condition (not a separate instruction in ladder), directly corresponding to negation in ST:

```
Ladder:  |--[/ ContactA ]----- ( Coil )--|
ST:      Coil := NOT ContactA;
```

Draw all three side by side on the board so the series/parallel/negated-contact distinction is visually anchored, not just verbally described.

5b. Guided Practice — 15 min

Build 2–3 rungs together as a class before releasing to independent work — deliberately extended over jumping straight to solo work, since ladder syntax is genuinely unfamiliar on first exposure. Suggested progression, narrated aloud while building each: 1. Single NO contact → coil 2. Two NO contacts in series (AND) 3. Two NO contacts in parallel (OR)

5c. Symbolic Addressing & Tag Naming — 15 min

Raw addressing vs. symbolic tags, with actual syntax:

- Raw: `%I0.0` , `%Q0.1` (IEC generic) — the literal physical terminal address, encoding module/channel position
- Symbolic: `Start_PB` , `Motor_Run` — a named tag mapped internally to that same physical address, but referenced by name in all logic

State the professional norm explicitly: production-grade programs are almost never written with raw addresses visible in the logic body — they use tags, because raw addresses carry no semantic meaning six months later; `%I0.3` tells a future maintainer nothing about what's physically wired there.

Establish a naming convention now, to hold for the rest of the week — e.g., PascalCase (`MotorRun`) or snake_case (`motor_run`), with a consistent verb/noun ordering (`Start_PB` for a start pushbutton input, `Motor_Run` for a run output). Write the chosen convention on the board as a standing rule.

5d. Commenting Rungs — 5 min

Show adding a rung comment/description in the software. Frame the distinction precisely: the rung's contact/coil structure shows *what* the logic evaluates; a comment records *why* — e.g., "Motor_Run requires seal-in per motor start/stop pattern; see Block 6" is a comment that saves real debugging time for someone reading the program cold later.

5e. Project: Two-Button AND/OR Logic — 10 min

```
AND version: Lamp := PB1 AND PB2;  
OR version:  Lamp := PB1 OR PB2;
```

Enforce tag usage throughout (per 5c's convention), not raw addresses — hold this requirement even under time pressure, since it's the first graded application of the naming convention established this block.

6. Day Project — Motor Start/Stop with Seal-In Circuit — 50 min

Core concept, stated before any software is touched: a momentary Start button must energize an output that remains energized after the button releases. This requires the output's own contact to be wired in parallel with the Start contact, so the output "holds itself in" once energized — combined with a Stop condition (wired NC per the fail-safe principle from Block 4d) that can break the circuit regardless of Start's state.

Logic pattern, in both ladder and ST — write this on the board and keep it visible the whole block:

```
Ladder:  
|--[ Start_PB ]-----+-----[/ Stop_PB ]----- ( Motor_Run )--|  
|                       |  
|--[ Motor_Run ]-----+ (seal-in contact, in parallel with Start_PB)  
  
ST:  
Motor_Run := (Start_PB OR Motor_Run) AND NOT Stop_PB;
```

State the mechanism precisely: on the scan Start_PB is pressed, Motor_Run's coil energizes. On the *next* scan, Motor_Run's own contact (now true, from the previous scan's output) is evaluated as true in the OR branch — meaning the rung stays true even if Start_PB has since released, because the parallel Motor_Run contact is now doing the work Start_PB was doing a moment ago. This is the seal-in mechanism, and it directly depends on the scan-cycle behavior established in Block 3 (this scan's output feeding next scan's input evaluation) — call out that connection explicitly.

Walkthrough structure:

- 1. Plan — 5 min:** sketch the seal-in rung on paper first, forcing reasoning about the pattern rather than trial-and-error clicking in software.
- 2. Wire I/O — 10 min:** Start button (NO), Stop button (NC per 4d's fail-safe principle), motor output (module type per 2c).
- 3. Write the rung — 15 min:** implement exactly from the paper sketch.
- 4. Test & debug — 15 min:** flag the common failure mode explicitly before it happens: omitting the Motor_Run seal-in contact entirely results in the motor running only while Start_PB is physically held — because without the parallel self-holding contact, there is nothing keeping the rung true once Start_PB releases. Watch for this specifically while circulating; if seen, ask "what condition is keeping this rung true after you let go of Start?" rather than fixing it directly.
- 5. Class review — 5 min:** surface the 2-3 most common mistakes actually observed, address as a group.

If finished early: add forward/reverse control with mutual-exclusion interlocking — two directional outputs that can never both be true simultaneously:

```
Fwd_Run := (Fwd_PB OR Fwd_Run) AND NOT Stop_PB AND NOT Rev_Run;  
Rev_Run := (Rev_PB OR Rev_Run) AND NOT Stop_PB AND NOT Fwd_Run;
```

State explicitly: each direction's rung includes a NOT of the *other* direction's own output — meaning even if both pushbuttons were somehow pressed simultaneously, whichever output happened to energize first on a given scan locks out the other on the very next scan. This exact interlocking pattern reappears formally in the safety-circuit content on Day 5 — flag this as deliberate forward-seeding, not a disconnected bonus exercise.